

TalkToDoc

Local Setup and Run Guide

Step-by-step instructions to get the application fully functional on a local machine.

Applies to: TalkToDoc_final.zip (final codebase)

What You Need Before Starting

The following must be installed on the machine before any commands are run. Install these once. They do not need to be repeated for future runs.

Python	3.11	<code>python3 --version</code>	python.org
pip	bundled with Python	<code>pip3 --version</code>	Included with Python
ffmpeg	Any recent	<code>ffmpeg -version</code>	ffmpeg.org / brew install ffmpeg / apt install ffmpeg
Git	Any	<code>git --version</code>	git-scm.com (optional, only needed to clone)
OpenAI API key	N/A	Check platform.openai.com	Required for live translation + NLU. Not needed if using MOCK_MODE.

ffmpeg is required by Whisper to read audio files. The app will import but audio transcription will fail if ffmpeg is missing.

Disk and Memory Requirements

These are confirmed figures based on the actual packages and models the codebase loads.

Python packages (pip install)	~6.5 GB	—
Whisper base model checkpoint	142 MB	~400 MB
YarnGPT-local (SmolLM2-360M backbone)	~724 MB	~750 MB
WavTokenizer checkpoint (.ckpt)	~300 MB	—
facebook/mms-tts-pcm (Pidgin TTS)	~100 MB	~200 MB
Flask process + overhead	—	~150 MB

TOTAL	~8.5 GB	~1.5 GB minimum 4 GB recommended

Ensure at least 10 GB free disk space before running pip install. pip keeps a download cache in addition to the installed files. Model checkpoints are downloaded automatically on first run and cached in ~/.cache/whisper and ~/.cache/huggingface/hub. They are not re-downloaded on subsequent runs.

Setup Steps

Follow these steps in order. Every command is shown exactly as it should be typed. The Where / Notes column says which terminal or application to use for each command.

Step 1 — Unzip the codebase

<code>unzip TalkToDoc_final.zip</code>	Terminal — run in the folder where the zip was saved
<code>cd TalkToDoc</code>	Terminal — enter the project folder

After unzipping, the folder structure should look like this: app.py, database.py, stt.py, tts.py, translation.py, nlu.py, requirements.txt, env, schema.sql, schema_postgres.sql, Dockerfile, Procfile, static/, templates/.

Step 2 — Create and activate a virtual environment

<code>python3 -m venv venv</code>	Terminal — inside the TalkToDoc folder
<code>source venv/bin/activate</code>	Terminal (Mac / Linux)
<code>venv\Scripts\activate</code>	Terminal (Windows)

You will see (venv) at the start of the terminal prompt when the environment is active. All pip install commands must be run with the environment active. Re-activate it with the same activate command any time you open a new terminal.

Step 3 — Install all Python dependencies

<code>pip install -r requirements.txt</code>	Terminal — inside TalkToDoc folder, venv active. This downloads ~3.2 GB and installs ~6.5 GB. Allow 10-30 minutes depending on connection speed.

This installs: Flask, OpenAI client, Whisper, PyTorch, torchaudio, transformers, YarnGPT, scipy, psycpg2-binary, gunicorn, and huggingface_hub. PyTorch and its NVIDIA CUDA libraries account for most of the size.

Step 4 — Configure the environment file

Open the file named env (no dot, no extension) in a text editor and fill in the values:

<code>nano env</code>	Terminal — or open with any text editor (Notepad, VS Code, etc.)	
SECRET_KEY	Any long random string. Example: <code>python3 -c "import secrets; print(secrets.token_urlsafe(32))"</code> generates one.	Always

MOCK_MODE	Set to true to run without an OpenAI API key. Translation returns the original text unchanged. NLU uses keyword matching. Full UI works. Good for testing all non-AI parts.	Optional
OPENAI_API_KEY	Your OpenAI API key from platform.openai.com. Not required when MOCK_MODE=true.	When MOCK_MODE is not true
PROVIDER_TOKEN	A long random string that the provider enters at /provider-login. If left unset (commented out), provider routes are unprotected. Fine for local dev and demos.	Optional
DATABASE_URL	Leave commented out for local development. SQLite is used automatically when this is absent.	Production only

Step 5 — Run the application

<code>python app.py</code>	Terminal — inside TalkToDoc folder, venv active. Starts Flask in development mode on port 5000.

The first time this runs, it will download the Whisper base model (142 MB) and the YarnGPT and MMS-TTS model checkpoints (~1.1 GB total). This only happens once. Subsequent starts load from the local cache and take roughly 20-60 seconds depending on hardware while the models load into memory.

You will see output like: Running on http://0.0.0.0:5000. The app is ready when this line appears and the terminal stops scrolling.

Step 6 — Open the app in a browser

<code>http://localhost:5000</code>	Browser — Patient interface
<code>http://localhost:5000/provider</code>	Browser — Provider interface
<code>http://localhost:5000/provider-login</code>	Browser — Provider login page (only appears if PROVIDER_TOKEN is set in the env file)

Step 7 — Test with a message

Confirm the app is working end-to-end:

1.	Open <code>http://localhost:5000</code> in the browser.	Browser
2.	Select a language (e.g. Yoruba).	Browser — patient page
3.	Type a short message and click Send to provider.	Browser — patient page
4.	Open <code>http://localhost:5000/provider</code> in a second browser tab.	Browser — provider page

5.	The message should appear in the Waiting queue.	Browser — provider page
6.	Click the message, type a reply in English, click Preview, then Send reply.	Browser — provider page
7.	Return to the patient tab. Click Check now. The reply should appear with an audio player.	Browser — patient page

Stopping and Restarting

<code>Ctrl + C</code>	Terminal — stops the running Flask server
<code>source venv/bin/activate</code>	Terminal — re-activate venv if you open a new terminal window
<code>python app.py</code>	Terminal — restart the server after stopping it

The SQLite database (talktodoc.db) persists between runs. All patient messages, sessions, and provider replies are retained. To start fresh, delete talktodoc.db and restart. It will be recreated automatically.

Running Without an API Key (MOCK_MODE)

MOCK_MODE lets you test the entire application without spending any OpenAI API credits. Set it in the env file before starting the server.

<code>MOCK_MODE=true</code>	env file — add or uncomment this line, then restart the server

What works in MOCK_MODE:

Translation	Returns the original text unchanged, or a labelled [MOCK] string when source and target languages differ.
NLU	Scans the text for keywords (headache, fever, cough, pain, etc.) and returns a pre-written mock summary.
STT	Works normally. Whisper runs locally and does not need an API key.
TTS	Works normally. YarnGPT and MMS-TTS run locally and do not need an API key.
All UI flows	Patient submit, provider queue, preview, reply, audio playback all work exactly as in production.

Common Problems and Fixes

ModuleNotFoundError	The virtual environment is not active. Run <code>source venv/bin/activate</code> (Mac/Linux) or <code>venv\Scripts\activate</code> (Windows), then run <code>python app.py</code> again.

KeyError: 'SECRET_KEY'	The env file is missing or SECRET_KEY is not set. Open the env file, set SECRET_KEY to any random string, and save.
ffmpeg not found / audio transcription fails	ffmpeg is not installed. Install it: brew install ffmpeg (Mac), sudo apt install ffmpeg (Ubuntu/Debian), or download from ffmpeg.org (Windows).
App starts but TTS produces no audio	The WavTokenizer checkpoint may have downloaded with an error. Delete ~/.cache/huggingface/hub and restart the app to re-download it.
Port 5000 already in use	Another process is using port 5000. Either stop that process, or set PORT=5001 in the env file and open http://localhost:5001 instead.
Provider page shows login form unexpectedly	PROVIDER_TOKEN is set in the env file. Enter the value you set there. To remove authentication, comment out or delete the PROVIDER_TOKEN line from the env file and restart.
First run is very slow (2-5 minutes)	The first run downloads Whisper base (142 MB), YarnGPT (724 MB), WavTokenizer (300 MB), and MMS-TTS (100 MB). This only happens once. Subsequent starts take 20-60 seconds while models load from the local cache.
openai.AuthenticationError	OPENAI_API_KEY is missing or wrong. Set the correct key in the env file, or set MOCK_MODE=true to run without it.

File Reference

Every file in the codebase and what it does.

<code>app.py</code>	Main Flask application. All routes, session handling, provider auth, audio cleanup.
<code>database.py</code>	Database access layer. Handles both SQLite (local) and Postgres (production) through the same interface.
<code>stt.py</code>	Speech-to-text. Loads Whisper base model and transcribes patient audio.
<code>translation.py</code>	Translation. Calls OpenAI gpt-5.6-terra to translate between patient language and English.
<code>nlu.py</code>	NLU. Calls OpenAI gpt-5.6-terra to summarise patient symptoms and intent. No diagnosis.
<code>tts.py</code>	Text-to-speech. YarnGPT for English/Yoruba/Igbo/Hausa. MMS-TTS for Nigerian Pidgin. Both local.
<code>env</code>	Environment configuration. Contains SECRET_KEY, OPENAI_API_KEY, MOCK_MODE, PROVIDER_TOKEN. Never commit this file.
<code>schema.sql</code>	SQLite schema. Creates app_user, session, and interaction tables.
<code>schema_postgres.sql</code>	Postgres schema. Same structure as schema.sql with Postgres-compatible syntax.
<code>requirements.txt</code>	All Python dependencies with exact version pins.
<code>Dockerfile</code>	Container definition for deployment. Pre-downloads model checkpoints at build time.
<code>Procfile</code>	Gunicorn start command for Render deployment.
<code>templates/base.html</code>	Base HTML template. Shared header, skip link, toast container.
<code>templates/patient.html</code>	Patient-facing UI. Language selector, voice/text input, transcript review, chat history.
<code>templates/provider.html</code>	Provider-facing UI. Queue, workspace, compose, preview, completed history.
<code>static/css/style.css</code>	All styles for both patient and provider interfaces.
<code>static/js/patient.js</code>	Patient interface JavaScript. Recording, submission, polling, chat bubbles.
<code>static/js/provider.js</code>	Provider interface JavaScript. Queue management, workspace, preview, send.
<code>static/js/shared.js</code>	Shared utilities used by both patient.js and provider.js.
<code>static/audio/</code>	Runtime directory. Preview and response audio files are saved here. Not in the zip, created automatically on first run.

Quick Reference — Daily Use

Once set up, these are the only commands needed for regular use.

cd TalkToDoc	Terminal — navigate to project folder
source venv/bin/activate	Terminal — activate virtual environment (Mac/Linux)
python app.py	Terminal — start the server
http://localhost:5000	Browser — patient interface
http://localhost:5000/provider	Browser — provider interface
Ctrl + C	Terminal — stop the server